Container Live Migration for Latency Critical Industrial Applications on Edge Computing

Keerthana Govindaraj
Corporate Sector Research and Advance Engineering
Robert Bosch GmbH
Renningen, Germany
keerthana.govindaraj@de.bosch.com

Alexander Artemenko
Corporate Sector Research and Advance Engineering
Robert Bosch GmbH
Renningen, Germany
alexander.artemenko@de.bosch.com

***Abstract*—A new level of factory automation demands processing vast amounts of data, complex orchestration of cyber-physical systems, and coordination of computation as well as communication resources in real-time. Virtualization and decentralized computation is becoming a de-facto solution for factory automation. Edge Computing (EC) is a promising approach to achieve the low latencies required by many industrial systems. It employs resource rich edge servers distributed within a factory that are placed close to end devices and assist them in executing computation intensive tasks and also in coordinating with each other. This paper discusses the requirements and challenges of EC for factory automation applications. In a distributed EC infrastructure, safe and timely operation of industrial applications require load balancing and mobility support and thus a seamless service migration between the edge servers. With the recent advances in virtualization and due to its advantages, virtual machine (VM) and container technologies are pavings its way into factory. Though containers have some distinctive advantages over VMs in EC, the service live migration has comparatively high downtime. This paper proposes a novel live migration scheme called redundancy migration that reduces the downtime by a factor of 1.8 compared to the stock migration in linux containers.**

***Index Terms*—containers, factory automation, downtime, latency, stateful, migration**

I. Introduction

The recently coined terms *smart factory*, *connected industry*, or *Industry 4.0* reflect the trend in manufacturing towards decentralized and highly interconnected cyber-physical production systems [1]. In future smart factories, machines and humans can cooperate on complicated tasks in real-time and production process reconfiguration will be highly flexible and fast. This will enable cost-effective production of small product series or even so-called *batch-size-one* products. Furthermore, the whole organizational value-chain can be highly coordinated with the production and vice versa. To achieve this new level of automation, production machines with their interfaces and functionalities have to be interconnected and mapped to cyber-physical systems with additional context information. Such systems with extensive complex orchestration require vast amounts of data to be transmitted, processed, and made available in real-time. Furthermore, small sensor devices present a limited computation and battery capacity and bigger devices with a dedicated hardware have limited flexibility in responding to the dynamically varying requirements. Virtualization of infrastructure resources and services offered by cloud data centers partially cater to the needs of such systems. However, due to their remote locations, the execution of many time-critical applications with response times requirements in terms of milliseconds is hard to achieve. Recently, *Edge Computing* (EC) concept has been studied extensively to address strict latency requirements of real-time applications, which can be neither achieved on the end devices due to computational complexity or hardware constraints [2] nor on the cloud due to unpredictable Wide Area Network latencies. EC constitutes *edge servers* (ES) placed between end devices and typically large and remote backend servers to execute time-critical tasks. Each edge server has a storage, computation, and a networking entity. Beside faster response times, EC can lead to reduced overall traffic load and energy consumption [3]. Additionally, processing the data locally within a factory keeps the data private and secure [4]. Fog and edge computing are two different terminologies that aim to bring the intelligence close to the devices to reduce the data traffic between the devices and the cloud. But the functionalities are executed on different levels of networking and computation entities. This paper considers EC with ESs placed within the factory premises and managed locally. However, EC differs from on-premises computing as it also offers agility in terms of re-provisioning, efficient resource utilization, device and location agnostic functioning, reliability, and scalability. Though EC inherits many advantages of cloud computing, the mobility and availability requirements of the industrial applications pose new requirements to the existing live migration schemes in terms of downtimes.

The goal and contribution of this paper is threefold: (i) it provides an overview of requirements and challenges of EC for factory automation applications in Section III; (ii) it illustrates the novel redundancy live migration scheme to reduce service downtime in Section V; (iii) it presents the evaluation of the benefits of redundancy migration scheme over the stock migration scheme in Section VI.

II. Factory Automation Applications

As introduced in Section I, factory automation consists of wide range of applications that pose different requirements. This section introduces the most prominent applications and

978-1-5386-7108-5/18/$31.00 ©2018 IEEE

their corresponding requirements regarding communication and computation resources.

A. Collaborative Robotics

One such application that assists the workers in performing mundane, painstaking or strenuous tasks is *collaborative robotics* (CR) [5]. CR refers to the concept of simultaneous human-robot collaboration. The safety requirements in industrial environment are very stringent and even more critical for collaborative robotics system in order to safeguard human workers [6]. Therefore, sensing, processing, and reacting must take place in real-time to realize safe collaboration. Input from multiple sensors such as cameras, microphones, indoor localization system, radars, lidars, inertial and proximity sensors need to be analyzed to decide and execute the next actions of a robot [7], [8]. Such control steps have to be executed in few millisecond cycles to avoid control loop delays and safety hazards.

B. Augmented Reality

Another application that is recently gaining popularity in industrial environment is *augmented reality* (AR) [9]. AR assists a worker by providing step-by-step instructions with easy visual aids and augmentations either to perform a new task or to repair a machine. To display these augmentations, a video frame captured from the image sensor in a camera needs to undergo object detection and feature detection followed by rendering of the object. These processing steps are computationally intensive [10] and additionally the end-to-end latencies must be less than 16 ms [11] to avoid perceptual issues.

C. Autonomous Transport System

Another application that eases the workers job in performing mundane tasks is *autonomous transport system* (ATS). ATS transports objects between production lines in a factory without or with very little human intervention. Therefore, the autonomous guided vehicles (AGV) require accurate localization using obstacle-based grid maps, feature-based metric maps, and topological maps generated using multiple sensors distributed in the factory environment [12]. Such complex data fusion algorithms need to work in real-time to avoid damages. Additionally, the AGVs need to coordinate together to meet the strict deadlines of the production system.

D. Predictive Maintenance

Supervisory Control and Data Acquisition (SCADA) is an established information system for collecting and analyzing real time data to monitor and control equipments in modern manufacturing industries [13]. Predictive maintenance is a recent advancement in manufacturing systems as it does not limit to just sensing and correction performed in SCADA, rather extends to prediction, optimization, and prevention [14]. This is mainly dependent on the big data environment collecting huge amount of data from every possible sensor in a factory. Therefore, the amount of data scales with the size of the plant and has an impact on the communication and computation resources required.

III. Special Challenges of EC for Factory Automation

The factory automation applications described in Section II present a new set of computation and communication requirements. The main challenge for EC is to guarantee the deterministic operation of industrial applications under such strict latency and reliability requirements, particularly for a dynamic and complex production systems. This section provides a detailed overview of the challenges factory automation poses towards EC.

A. System requirements

On the one hand, applications in factory automation often require *safe and reliable operation* with hard real-time deadlines, high degree of determinism, and availability of the communication system [15], [16]. A faulty or delayed operation of a device in a factory can lead to fatal consequences. For example, in certain closed-loop control applications, the period of a control cycle must be as low as 0.5 ms, and the packet loss rate must be below 10^{-9} [17]. On the other hand, a *huge amount of data processing* is required. The data generated and collected for machine-learning models for predictive maintenance needs to be processed on a powerful server [14]. This implies that the EC system must be capable of handling both latency requirement and huge data generated simultaneously.

B. Infrastructure requirements

The *size of a manufacturing plant* can vary from a few hundred square meters for a small local workshop to one million square meters as in Tesla's Gigafactory [18]. Respectively, the number and the complexity of interacting applications, humans, end devices, and machines scales with the factory size. With a centralized factory cloud, the applications generating huge amount of data cause a bottleneck which demands for high orchestration overhead to be able to cater to the latency critical applications. Moreover, restructuring the whole factory network to install high datarate components is expensive and time consuming. Therefore, large factories require an infrastructure of well orchestrated distributed edge servers placed close to the end devices to interconnect a large number of applications [19]. And since each production step is dependent on the adjacent processes, a tight interconnection of individual edge servers, their resources and running applications is important.

C. Operating environment requirements

Virtualization supports flexibility in terms of hardware, isolation, and scalability. This has attracted a large amount of work proposing novel solutions in virtualization technologies. But the virtualization layer adds additional delay to the response times. Virtual Machines (VM) and containers are both well known virtualization technologies. Containers is recently gaining popularity due to its low virtualization overhead and fast responsiveness. That implies, more containers hosting services can be deployed on a resource constrained edge servers.

Moreover many recent works [20], [21] have compared the performance of a VM technology with the recent container technologies to show that containers have almost near-native performance. Farris et al. [22] prove feasibility of container technology for latency critical applications and for applications requiring huge data processing. Thus, container seems to be an attractive candidate for industrial applications. However, the orchestration of container resources and management of the running applications on the container are still open challenges in EC.

D. Service migration

Brettel et al. [19] also emphasize how the infrastructure decentralization and virtualization is becoming a default solution for factory automation. But the devices such as AR and AGV are inherently mobile. Moreover, a dynamic factory environment demands flexible reconfiguration and load balancing. Altogether, it calls for an intelligent service migration from the current ES to a next suitable ES. Section III-A presents that the response time requirements can be as low as 0.5 ms. Thus the service migration must take place with interruption times lower than this limit. On the one hand, Section III-C presents that containers have distinctive advantages over VMs and on the other hand Theimer et al. [23] show that the efforts involved in migrating a process reduces with the increased level of isolation and separation. Currently the downtime experienced in container technology is high and it needs to be further reduced to perform service migration in factory automation scenario.

IV. Live Migration

Live migration is the concept of moving a service, independent of virtualization environment, from one server to another without losing the state of the application running. This involves migration of CPU, memory, and network state. The destination server requires the CPU, memory, and network states to restore the service correctly. During this process, the memory state keeps changing and the rate at which the state changes and the size of the state depends on the application.

The process of live migration is measured by two important metrics, namely *downtime* and *migration time*. Downtime corresponds to the duration in the migration process during which the service is completely halted. During this period, the service is unavailable and thus influences the service level agreement (SLA) of an application. Downtime is an important metric because, downtimes higher than the response times required by the industrial applications can jeopardize the safety of humans working. Migration time corresponds to the complete duration from the trigger of migration until the service is restored on the destination server and starts running. At this point the source server can shut down the service and no further actions need to be taken place [24]. During this period of migration the CPU and memory resources are occupied on both the source and destination servers along with network resources connecting them. A scenario with multiple live migration processes and with long migration times in a system can reduce the efficient usage of resources. However, the main focus of this paper is to reduce the downtime and the following schemes are examined regarding the downtime.

A. Basic Live Migration Schemes

1) Pre-copy migration: Theimer et al. [23] proposed this scheme for the migration of CPU and memory state. This scheme consists of two phases, namely *push phase* and *stop-and-copy phase*. During the push phase, the source server copies the memory corresponding to the service iteratively as and when the memory page is altered and transfers to the destination server. This phase continues until the number of altered memory pages to be transferred reduces to a certain size or the number of iterations exceeds a certain threshold. Now the source server initiates the stop-and-copy phase. In this phase the source server freezes the service shortly to obtain a consistent state, which results in a critical downtime of the service and is unavailable for the client. On receiving this final information of the memory state, the destination server restores and resumes the service and concludes the migration process.

2) Post-copy migration: Unlike pre-copy migration, this scheme is a complete opposite approach also proposed by Hines et al. [25] to reduce the downtime. This scheme consists of *stop-and-copy phase* and *pull phase*. During the stop-and-copy phase, the source server freezes the service to transfer only the CPU state to the destination server and the client experiences downtime. Once the destination server receives the CPU state, it resumes the service. However, when it starts to run, it experiences pagefaults on every memory page request as it is not yet transferred from the source server. Now the destination server starts the pull phase, in which it pulls the requested memory pages from the source server. Thus, during this phase the service is jittery and the response times are hard to predict and the client experiences a degraded service performance.

3) Hybrid: Since pre-copy and post-copy has complementing performances, a combination of both of these schemes results in a good trade-off [26]. Thus, the combination results in three phases, namely *push phase*, *stop-and-copy*, and *pull phase*. During the push phase, the source server transfers an image of all the states to the destination server as in pre-copy migration scheme. As a next step the source server executes the stop-and-copy phase which leads to a service downtime as in previous schemes. Finally the destination server restores the service and requests the missing memory pages during the pull phase.

Discussion

The main objective of all these schemes is to reduce the downtime to a minimal level. In the pre-copy migration scheme, the downtime depends on the size of the memory state that needs to be transferred during the stop-and-copy phase. An advantage of this scheme is that, it has minimal influence on the performance of the service during the migration [25]. Moreover, during this process if the destination server fails to

restore, it does not influence the source server and also the client continues to avail the service. However, the main idea is to reduce the downtime, while the migration time depends on the threshold limits set and on the service running.

Compared to the pre-copy migration, post-copy migration solves the problem of non-converging working sets and has smaller downtime as the CPU state is smaller than the memory state. However, the client suffers from the jittery performance of the service during the pull phase. Moreover, since the source server suspends after the stop-and-copy phase, there is no fail-safe mechanism for the destination server in case it fails to resume the service successfully.

In the hybrid migration scheme, downtime as well as migration time can be optimized based on the requirements, nevertheless the stop-and-copy phase is still inevitable.

B. Live Migration Optimization Schemes

Since all the basic live migration schemes result in an indeterministic downtime, many scientific groups have attempted to optimize the process.

1) Optimization of Basic Live Migration Schemes: The following work propose optimization to the existing basic live migration schemes to reduce downtime.

Sharma et al. [27] propose a Three Phase Optimization for pre-copy migration. In the first phase, a page selection mechanism transfers all the memory pages in the first iteration. In the second phase, an algorithm identifies the Writable Working Set (WWS). It excludes the pages belonging to the WWS to prevent transferring the same page multiple time as it could be modified again in the next iteration. The authors claim to reduce the downtime by 3% for higher workload by using run-length encoding for the last iteration step.

Hines et al. [25] propose an optimization of post-copy migration to reduce jitter during the pull phase. Their pre-paging mechanism predict which pages will be accessed in the future and they can be fetched proactively and the client does not experience any page faults.

There is a lot of work available that optimizes the hybrid migration scheme as it facilitates to obtain a trade-off between the downtime and the migration time but still requires the stop-and-copy phase.

2) State Reproduction Schemes for Migration: As seen from the existing work, even the optimization of basic live migration schemes cannot avoid a brief stop-and-copy phase to get a consistent state. Thus, a novel live migration approach is required to address the low downtime requirement. One such approach towards this problem is the trace and replay method. In this method, the recorded transaction from a particular state on the source server is replayed on the destination server. Liu et al. [28] propose a combination of checkpoint/recovery and trace/replay methods to reduce the downtime. In this paper, they use a checkpointing procedure based on the copy-on-write mechanism. On triggering the checkpoint, a copy of VM is made and the VM continues to run on the source server. From this point onwards, the source server starts to trace and log the operations of the VM. Liu et al. build upon ReVirt [29] to log only the asynchronous interrupts to keep the logging overhead within reasonable bounds. The source server transmits the checkpoint to the destination server. Once the destination server restores the VM from the checkpoint and resumes the service, the source server transmits the logs of the source VM. On replaying the logs, the VM on the destination server can synchronize to the current state of the VM on source server. They conclude the migration process with a brief stop-and-copy phase once the log file size falls under a certain threshold. Though this approach is a novel fault-tolerance scheme, a brief stop-and-copy phase is still inevitable.

Jiang et al. [30] propose another such high availability approach using checkpointing and replay. They provide a fault-tolerant system by checkpointing the system in regular intervals and replicating the state to a backup system. Since in their approach, checkpointing cause a certain downtime, they reduce the frequency of creating checkpoints and rather use input replay to recover intermediate state. However, their approach highly rely on the infrastructure of Xen hypervisor.

However, Section III presented a need for low response times in industrial applications which emphasizes the importance of containers over VMs and also the need for live migration schemes with downtimes within acceptable limits. None of the existing approaches address these requirements.

V. Redundancy Migration

Unlike the basic live migration schemes (cf. Section IV-A), redundancy migration does not execute a *stop-and-copy* that results in downtime due to creation, transmission, and restoration of a consistent state image on the destination server. Furthermore, unlike the existing live migration optimization schemes (cf. Section IV-B), this scheme is application agnostic, environment independent and also suitable for TCP or UDP applications. However, it is based on the assumption that the application is deterministic, and the processing speed of the network packets on the destination server is higher than the packet generation rate and transmission rate of the packets on the client. Most importantly, this paper evaluates the redundancy migration scheme on the linux containers.

A. Basic Idea

Fig. 1 illustrates the redundancy migration scheme. Section III described that, the trigger for live migration can be the mobility of the devices, need for server maintenance or load balancing requirement in the system. In any case, the destination server is chosen by an *edge controller* using a management scheme that is not discussed here as it lies out of the focus of this paper. From the time instance when the migration is triggered, the client transmits the packet streams to the destination server. The destination server buffers these packets and forwards the same to the source server. Meanwhile, the source server starts to iteratively transmit the files relevant to the service to be migrated to the destination server. Additionally, it transmits a checkpoint of CPU, network, and memory state to the destination server while the

service continues to run on the source server. In parallel, the destination server restores the checkpoint provided by the source server. Once the service is up and running, the destination server starts to replay all the buffered packets. However, the WWS continues to change which depends on the service running and hence diverging the states of the service on the source and destination servers. Finally once the packet buffer is empty, the application state on the source and destination server shall be the same.

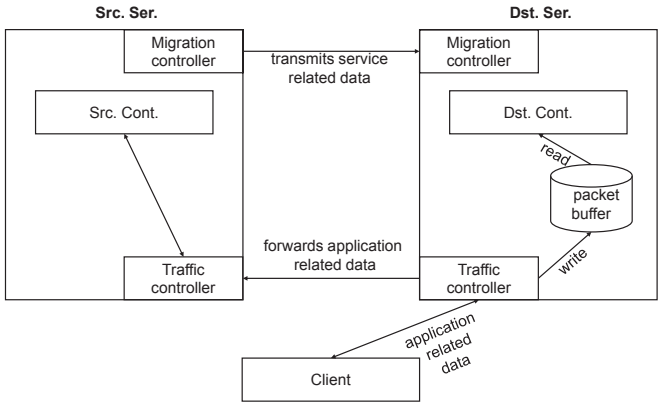


Fig. 1: Redundancy migration high-level architecture

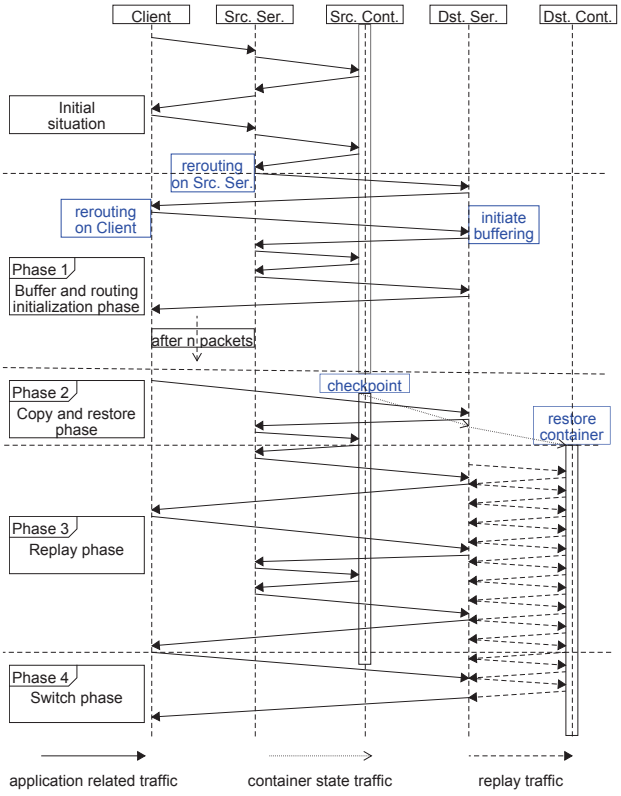


Fig. 2: Redundancy migration sequence diagram

B. Design and Implementation

Redundancy migration approach is evaluated using LinuX Containers (LXC). They are also referred to as lightweight virtualization as they have very low virtualization overhead [31]. They are directly implemented on the Linux OS and thus can only execute applications that are compatible with the host OS. Though they lack a virtualization layer, they can provide a certain level of separation by using namespaces. They use control groups (cgroups) [32] to account and monitor the resource utilization. The live migration process requires the following namespaces: Mount namespaces; UTS namespaces; PID namespaces. Linux containers Daemon (LXD) is a management tool for LXCs provided by Canonical [33] and uses `liblxc` library to manage these containers. It also provides an interface to *Checkpoint/Restore in Userspace (CRIU)*, a tool that provides a mechanism to extract the state, called *checkpoint*, of a Linux process and restore it [34]. The checkpoint consists of CPU context, memory, network state, etc. of a process running on the Linux kernel.

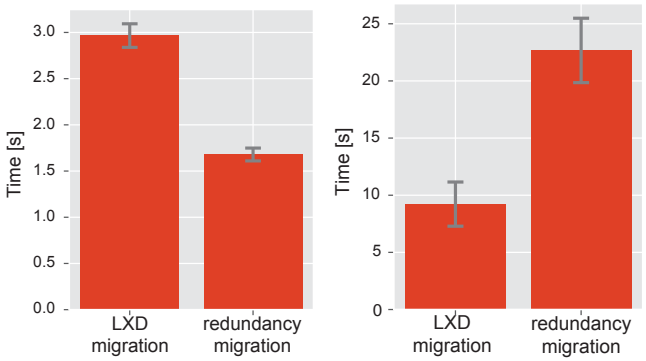
The redundancy migration constitutes four phases. Migration controller and traffic controller installed on the source and destination servers assist the migration process. Fig. 2 illustrates the the steps involved in each phase and are described below in detail.

1) Buffer and routing initialization phase: Initially the *Client* communicates with the source container (*Src. Cont.*) running on source server (*Src. Ser.*). When the *edge controller* triggers live migration, the client starts to transmit its packets to the destination server (*Dst. Ser.*). The *traffic controller* on the *Dst. Ser.* initializes the *packet Buffer* and simultaneously forwards these packets to the *Src. Ser.*. The *traffic controller* on the *Src. Ser.* forwards the response to the client via the *Dst. Ser.*.

2) Copy and restore phase: This phase starts when the *edge controller* makes a clone of the *Src. Cont.* by triggering *LXD* on *Src. Ser.* to start the snapshot procedure. *CRIU* creates a checkpoint to obtain the consistent state of the process. The *migration controller* on the *Src. Ser.* copies the snapshot and the checkpoint to the *Dst. Ser.*. The *migration controller* on the *Dst. Ser.* resumes the container and the process by restoring the snapshot and the checkpoint.

3) Replay phase: Once the *Dst. Ser.* successfully resumes the destination container (*Dst.Cont*), the *migration controller* sets up the networking interfaces. Now the *traffic controller* on the *Dst. Ser.* starts to replay the packets exactly from the instance a checkpoint was created in order to catch up to the state of the *Src. Cont.*. Furthermore, it constantly compares the output of the *Dst.Cont* and *Src. Cont.*.

4) Switch phase: Once the packet buffer is empty, the *migration controller* on the *Dst. Ser.* compares both the outputs from *Dst.Cont* and *Src. Cont.* again. If the outputs are same, it declares that the *Dst.Cont* is ready to take over. Thus, the *traffic controller* on the *Dst. Ser.* stops forwarding the packets



(a) Downtime (b) Migration time

Fig. 3: Downtime and migration time comparison in standard LXD migration and redundancy migration

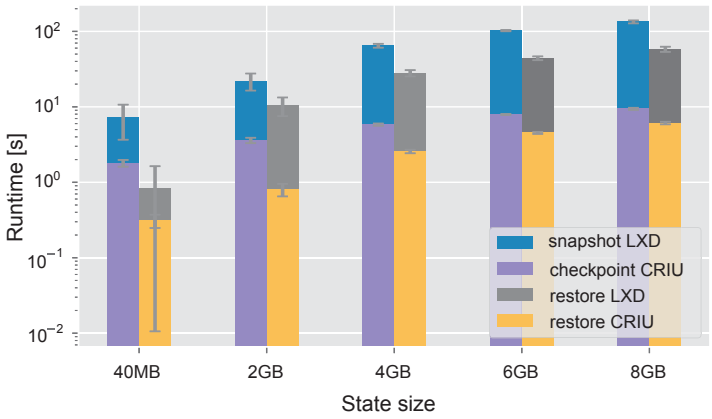


Fig. 4: Influence of state size on checkpoint and restore times

to the *Src. Ser.* and indicates the *migration controller* on the *Src. Ser.* to shutdown the *Src. Cont.*.

VI. Evaluation

This section provides the comparison of *redundancy migration* against LXD stock live migration in terms of the downtime and migration time. Moreover, it is important to evaluate the additional delay contributed due to rerouting the packets which is the extra overhead that *redundancy migration* introduces. The set-up consists of two hosts running a Linux kernel version `4.13.0-rc7` with Ubuntu `16.04` connected via a patch cable. Both the hosts support a modified version of *LXD* `2.17` environment and unmodified version of *CRIU* version 3.4. The *Src. Ser.* initially has a *Src. Cont.* running 8 processes. In addition to it, client-server runs a process that performs sequential incrementing operation and is considered to illustrate redundancy live migration.

A. Downtime and Migration Time

Section III-A has emphasized that downtime plays a critical role in a factory automation scenario. Fig. 3 represents the downtime and the migration time in *LXD* stock migration and redundancy migration. The plots show an average over 30 runs and have a confidence interval of 99.9 %. Fig. 3a shows, the redundancy migration attains an average downtime of $\approx$ 1.68 s when compared to the LXD stock migration resulting in $\approx$ 2.97 s and shows an improvement by a factor of $\approx$ 1.8. The main reason for the downtime in redundancy migration scheme is the checkpoint creation time required by `CRIU` whereas in the stock live migration it consists of snapshot and checkpoint creation, transfer, and restoration. Redundancy migration scheme can strongly benefit further by the improvement in the checkpoint process in `CRIU`. Furthermore, the benefit of redundancy migration increases with the growth in size of the application state. A benchmarking tool called `lookbusy` [35] is used to control the state size in order to quantify the benefit. Fig. 4 shows that the checkpoint and restore times rise logarithmically with the increasing state size.

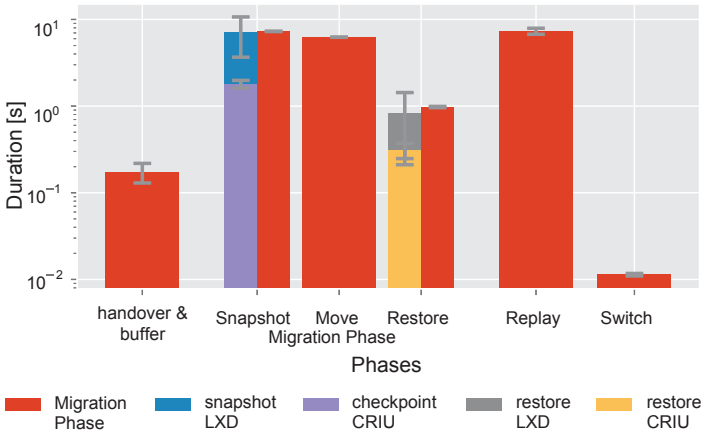


Fig. 5: Migration phases and time

However, redundancy migration introduces some overhead that influences the migration time. Fig. 3b shows an increase in the migration time by a factor of $\approx$ 1.7 when compared to the LXD stock live migration.

B. Migration Phases

Though downtime plays an important role in providing low response times, migration time influences the occupancy of the resources and thus also need to be kept low. This section provides a comparison of time duration of the phases involved in the complete migration process, namely *buffering and rerouting initialization*, *copy and restore*, *replay*, and *switch* phases. Fig. 5 shows that a long replay phase is the main reason for an increased migration time. This is directly proportional to the number of packets buffered during the *copy and restore* phase. Therefore, the total migration time can be reduced by optimizing the *copy and restore* phase.

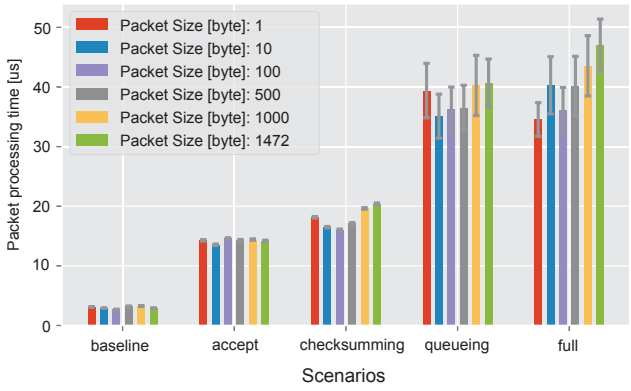


Fig. 6: Rerouting overhead

C. Rerouting Overhead

Redundancy migration approach adds an additional overhead on the packet transmission. Rerouting overhead is an important performance indicator as this latency is added to each packet during the migration. This in turn can influence the latency requirements of the industry automation applications. The packets are tracked on the *Dst. Ser.* in order to measure the overhead.

Fig. 6 represents five measurement series to quantify the overhead for packets of different sizes. The baseline scenario measures a completely unmodified packet processing time. The accept scenario measures the time required by the routing table rules to queue the packets and hand off a copy of each packet to the userspace and the userspace in turn to command the kernel to accept the packet. The checksumming scenario measures the time required to calculate a checksum for each packet before accepting. The queueing scenario measures the time required to add a copy of the packet to the *Packet Buffer* before telling the kernel to accept the packet. The full scenario considers all the above mentioned processes together. The overhead of processing times measures $\approx 30\,\mu$s to $\approx 45\,\mu$s whereas for baseline it varies between $(2.5\,\mu\text{s} - 3.5\,\mu\text{s})$. For the given network setup the overhead measured results in the end to end latency of $100\,\mu$s. Since the overhead is in μs and as it does not vary for the packets of different sizes, it has very little effect on the latency requirements.

VII. Conclusion and Future Work

This paper presents the most prominent factory automation applications with their unique computation and communication requirements. Edge computing (EC) is identified as a promising approach to address these requirement and the new challenges faced by EC in factory automation are elaborated in detail. The distributed edge server infrastructure as well as mobility support, load-balancing, and low response time requirements demand for a novel live migration scheme with low downtime. Though containers are gaining popularity as lightweight virtualization technologies in EC, the downtimes experienced in live migration are still high for factory automation. The redundancy migration scheme proposed in this paper reduces the downtime in the containers by a factor of 1.8 when compared to LXD stock live migration. This could be further reduced by optimizing the tool used (*CRIU*) to create the checkpoint.

References

[1] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, 2015.

[2] N. Fernando, S. W. Loke, and W. Rahayu, "Mobile cloud computing," *Future Gener. Comput. Syst.*, vol. 29, no. 1, jan 2013.

[3] M. T. Beck, S. Feld, A. Fichtner, C. Linnhoff-Popien, and T. Schimper, "ME-VoLTE: Network functions for energy-efficient video transcoding at the mobile edge," in *Proceedings of the International Conference on Intelligence in Next Generation Networks (ICIN)*, Feb. 2015.

[4] A. Papageorgiou, B. Cheng, and E. Kovacs, "Real-time data reduction at the network edge of internet-of-things systems," in *Proceedings of the International Conference on Network and Service Management (CNSM)*, Nov 2015.

[5] J. Krueger, T. Lien, and A. Verl, "Cooperation of human and machines in assembly lines," *CIRP Annals - Manufacturing Technology*, vol. 58, no. 2, 2009.

[6] *ISO/TS 15066, Robots and robotic devices-Collaborative robots*, The International Organization for Standardization Std., 2016.

[7] K. P. Hawkins, N. Vo, S. Bansal, and A. F. Bobick, "Probabilistic human action prediction and wait-sensitive planning for responsive human-robot collaboration," in *Proceedings of the IEEE-RAS International Conference on Humanoid Robotics (Humanoids)*, Oct. 2013.

[8] A. W. Stroupe, M. C. Martin, and T. Balch, "Distributed sensor fusion for object position estimation by multi-robot systems," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, 2001.

[9] V. Paelke, "Augmented reality in the smart factory: Supporting workers in an industry 4.0 environment," in *Proceedings of the IEEE Emerging Technology and Factory Automation (ETFA)*, Sep. 2014.

[10] P. Hasper, N. Petersen, and D. Stricker, "Remote execution vs. simplification for mobile real-time computer vision," in *Proceedings of the International Conference on Computer Vision Theory and Applications (VISAPP)*, vol. 3, Jan. 2014.

[11] S. R. Ellis, K. Mania, B. D. Adelstein, and M. I. Hill, "Generalizeability of latency detection in a variety of virtual environments," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 48, no. 23, pp. 2632–2636, 2004.

[12] H. Cheng, H. Chen, and Y. Liu, "Topological indoor localization and navigation for autonomous mobile robot," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 729–738, April 2015.

[13] . H. A. S. I. Shaheen, "Future scada challenges and the promising solution: the agent-based scada," *International Journal of Critical Infrastructures*, 2014.

[14] J. Lee, H. D. Ardakani, S. Yang, and B. Bagheri, "Industrial big data analytics and cyber-physical systems for future maintenance & service innovation," *Procedia CIRP*, vol. 38, 2015.

[15] C. E. Pereira and P. Neumann, "Industrial communication protocols," in *Springer Handbook of Automation*, 2009.

[16] *Guideline VDI/VDE 2185, Radio based communication in industrial automation*, VDI/VDE Std., Dec. 2009.

[17] A. Frotzscher, U. Wetzker, M. Bauer, M. Rentschler, M. Beyer, S. Elspass, and H. Klessig, "Requirements and current solutions of wireless communication in industrial automation," in *Proceedings of the IEEE International Conference on Communications Workshops (ICC)*, Jun. 2014.

[18] "Tesla gigafactory," 2016. [Online]. Available: https://www.tesla.com/gigafactory

[19] M. Brettel, N. Friederichsen, M. Keller, and M. Rosenberg, "How Virtualization, Decentralization and Network Building Change the Manufacturing Landscape: An Industry 4.0 Perspective," *International journal of mechanical, aerospace, industrial and mechatronics engineering*, vol. 8, no. 1, pp. 37–44, 2014.

[20] . Kovcs, "Comparison of different linux containers," in *International Conference on Telecommunications and Signal Processing (TSP)*, July 2017, pp. 47–51.
[21] R. Morabito and N. Beijar, "Enabling data processing at the network edge through lightweight virtualization technologies," in *IEEE International Conference on Sensing, Communication and Networking (SECON Workshops)*, June 2016, pp. 1–6.
[22] I. Farris, T. Taleb, A. Iera, and H. Flinck, "Lightweight service replication for ultra-short latency applications in mobile edge networks," in *IEEE International Conference on Communications (ICC)*, May 2017, pp. 1–6.
[23] M. M. Theimer, K. A. Lantz, and D. R. Cheriton, "Preemptable remote execution facilities for the v-system," *SIGOPS Oper. Syst. Rev.*, vol. 19, no. 5, pp. 2–12, Dec. 1985.
[24] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in *Proceedings of the Conference on Symposium on Networked Systems Design & Implementation - Volume 2*, ser. NSDI'05. USENIX Association, 2005, pp. 273–286.
[25] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy Live Migration of Virtual Machines," *SIGOPS Oper. Syst. Rev.*, vol. 43, no. 3, pp. 14–26, Jul. 2009.
[26] Z. Lei, E. Sun, S. Chen, J. Wu, and W. Shen, "A novel hybrid-copy algorithm for live migration of virtual machine," *Future Internet*, vol. 9, no. 3, 2017.
[27] S. Sharma and M. Chawla, "A three phase optimization method for precopy based vm live migration," *SpringerPlus*, vol. 5, no. 1, pp. 1–24, 2016.
[28] H. Liu, H. Jin, X. Liao, L. Hu, and C. Yu, "Live Migration of Virtual Machine Based on Full System Trace and Replay," in *Proceedings of the ACM International Symposium on High Performance Distributed Computing*, ser. HPDC '09. ACM, 2009, pp. 101–110.
[29] G. W. Dunlap, S. T. King, S. Cinar, M. A. Basrai, and P. M. Chen, "Revirt: Enabling intrusion analysis through virtual-machine logging and replay," *SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 211–224, Dec. 2002.
[30] B. Jiang, B. Ravindran, and C. Kim, *Lightweight Live Migration for High Availability Cluster Service*. Springer Berlin Heidelberg, 2010, pp. 420–434.
[31] R. Morabito, J. Kjllman, and M. Komu, "Hypervisors vs. lightweight virtualization: A performance comparison," in *IEEE International Conference on Cloud Engineering*, March 2015, pp. 386–393.
[32] "CGROUPS," https://www.kernel.org/doc/Documentation/cgroup-v1/cgroups.txt, accessed: 2017-09-04.
[33] "LXD," https://linuxcontainers.org/lxd/, accessed: 2017-10-14.
[34] "CRIU (Checkpoint/Restore in Userspace," https://criu.org/, accessed: 2017-10-14.
[35] Devin Carraway, "lookbusy – a synthetic load generator," http://www.devin.com/lookbusy/, accessed: 2017-11-02.